Burst-Aware Adaptive Round Robin: A Learned CPU Burst Predictor for Dynamic Time Quantum Selection

Kunal Srivastava
24BCE2476
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*—Round robin scheduling gives every process a fair share of the processor, but its behavior depends heavily on one fixed number, the time quantum. A quantum that is too small wastes time on context switches, and one that is too large makes the scheduler behave like first come first served. Many dynamic quantum schemes have been proposed, yet most of them compute the quantum from burst times that an operating system does not actually know before a process runs. This review document presents the problem, the related literature and the idea of a burst-aware adaptive round robin scheduler in which the next processor burst of each process is predicted from its recent execution history by a lightweight learned model, and the predicted value is then used to choose a bounded per-process quantum. The document covers the problem statement, motivation, objectives, existing approaches, a literature survey of eighteen publications, the research gap, the proposed methodology with an initial algorithm, the planned data sources, the evaluation parameters and the expected outcomes. Implementation and results are outside the scope of this first review.**

Keywords—operating systems, CPU scheduling, round robin, dynamic time quantum, burst time prediction, machine learning

I. Introduction

The short-term scheduler decides which ready process gets the processor next and for how long. This decision is taken thousands of times per second, so even a small improvement in it is felt across the whole system. Classical policies were designed around clear goals: rate-monotonic and earliest-deadline-first scheduling give guarantees for periodic real-time tasks [1], while lottery scheduling shares the processor in proportion to the tickets held by each client [2]. General-purpose systems mostly rely on time slicing, and round robin (RR) is the simplest and most widely taught form of it.

In RR every process runs for at most one time quantum and is then moved to the back of the ready queue. The policy is starvation free and gives good response time, but the quantum is normally a constant chosen by the designer. Real workloads are a mixture of short interactive bursts and long compute-bound bursts, so a single constant cannot suit all of them. Production schedulers have grown complicated in trying to handle this variety, and a study of the Linux scheduler showed that hand-tuned heuristics can hide performance bugs for years [3]. This has encouraged researchers to ask whether parts of the operating system can learn their parameters from observed behavior instead of having them fixed in advance [4].

This document is prepared for the first review of the project. It follows the order required for a conference-oriented project: Section II states the problem, Sections III and IV give the motivation and objectives, Section V describes existing approaches, Section VI is the literature survey, Section VII identifies the research gap, Section VIII describes the proposed solution, architecture, algorithm, data and evaluation parameters, and Section IX lists the expected outcomes.

II. Problem Statement

The performance of round robin scheduling is controlled by a time quantum that is fixed, while the processor bursts of real processes vary over several orders of magnitude and change during the lifetime of a process. Dynamic quantum algorithms try to adapt the quantum, but they take the burst time of every process as an input that is known beforehand. An actual operating system has no such information at dispatch time. The problem addressed in this project is therefore the following: how can the scheduler estimate the next processor burst of a ready process from information that is really available at run time, and use that estimate to choose a time quantum that reduces waiting time, turnaround time and the number of context switches without causing starvation or noticeable prediction overhead.

III. Motivation

- A context switch does no useful work. When the quantum is slightly shorter than a burst, the process is preempted just before it would have finished and has to wait for a full round to complete a small remainder.
- Dynamic quantum algorithms report good improvements, but they are tested on small hand-made process tables with known burst times [5], [6], [7]. It is unclear how much of that improvement remains when the burst times have to be estimated.
- Machine learning has already been used successfully to predict job running times in batch and cluster systems [8], [9], which suggests that execution history carries enough signal for prediction at the process level as well.
- The topic connects directly to the scheduling experiments done in the operating systems laboratory, so the baseline algorithms are well understood and the new idea can be compared with them fairly.

IV. Objectives

- To study fixed and dynamic quantum round robin algorithms and identify the assumptions they make about burst times.
- To build a labelled data set of per-process CPU bursts from scheduler traces and synthetic workloads.
- To design a lightweight predictor that estimates the next CPU burst of a process from its recent history and to compare it with exponential averaging.

- To design a quantum controller that converts the predicted burst into a bounded per-process quantum and prevents starvation.
- To evaluate the complete scheduler in a trace-driven simulator against first come first served, shortest job first, fixed quantum RR and published dynamic quantum RR algorithms.

V. Existing Systems and Approaches

A. Fixed Quantum Round Robin

The standard algorithm uses one quantum for all processes. It is simple to implement with a timer interrupt and a circular queue, and it bounds the response time of every process by the product of the quantum and the number of ready processes. Its weakness is that turnaround time and the context switch count depend strongly on how the quantum compares with the burst lengths of the current workload.

B. Shortest Job First with Exponential Averaging

Shortest job first gives the minimum average waiting time, but it needs the length of the next burst. The usual estimate is an exponential average of earlier bursts,

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n \tag{1}$$

where t_n is the measured length of the nth burst, τ_n is the previous estimate and α lies between 0 and 1. The estimate uses only one feature, the burst history of the same process, and reacts slowly when a process changes its behavior. It is used in this project as the baseline predictor.

C. Dynamic Quantum Round Robin

These algorithms recompute the quantum from the burst times of the processes that are currently in the ready queue, typically from their median [5], [6] or their mean [7]. They are discussed in detail in Section VI because they are the closest existing work to the proposed scheduler.

D. Proportional Share and Priority Based Schedulers

Lottery scheduling [2] and the schedulers used in current kernels distribute processor time according to weights or priorities, not according to burst length. They adapt the share of each process but still depend on tunable constants, and their growing complexity is itself a source of errors [3].

VI. Literature Survey

A. Classical Scheduling Theory

Liu and Layland [1] analysed fixed priority and deadline driven scheduling of periodic tasks and derived utilization bounds under which all deadlines are met. Their work shows the value of scheduling policies whose behavior can be reasoned about, a property that any learned scheduler should try to keep. Waldspurger and Weihl [2] introduced lottery scheduling, a randomized mechanism in which the probability of being chosen is proportional to the number of tickets held, giving flexible control over relative execution rates. Lozi et al. [3] examined the Linux scheduler on multicore machines and found several bugs that left cores idle while threads were waiting, which shows how difficult it is to maintain hand-written heuristics as hardware and workloads change.

B. Dynamic Time Quantum Algorithms

Matarneh [5] proposed a self-adjusting RR in which the quantum is recalculated in every round from the median burst time of the processes present in the ready queue. Behera et al. [6] also used the median, and in addition rearranged the ready queue so that short and long processes are served alternately. Noon et al. [7] replaced the median by the mean of the remaining burst times and recomputed it whenever the contents of the ready queue changed. All three papers report lower average waiting time, lower turnaround time and fewer context switches than fixed quantum RR. However, the evaluations use a handful of processes whose burst times are given as input, and none of the papers explains how these values would be obtained in a running system.

C. Prediction of Execution Time

Negi and Kumar [10] were among the first to apply machine learning to process scheduling in Linux. They used attributes of a program to predict its processor requirement with decision tree and nearest-neighbor classifiers and passed the prediction to the scheduler, obtaining a reduction in turnaround time. Their model works per program, not per burst. In parallel job scheduling, Tsafrir et al. [8] showed that replacing the run time estimates supplied by users with simple predictions derived from the recent jobs of the same user improves backfilling. Gaussier et al. [9] extended this idea with an online regression model and a loss function that reflects the unequal cost of under-prediction and over-prediction. Both studies confirm that history-based prediction is useful to a scheduler, although they deal with whole batch jobs that run for minutes or hours and not with processor bursts of a few milliseconds.

D. Learning Based Schedulers

Mao et al. [11] formulated multi-resource job scheduling as a reinforcement learning problem and trained a neural network policy that performed comparably to hand-designed heuristics. Their later system for data processing clusters [12] combined reinforcement learning with a graph neural network to schedule jobs made of dependent stages. Delimitrou and Kozyrakis [13] used classification techniques to infer the resource needs and interference sensitivity of incoming workloads so that the cluster manager can allocate resources without user reservations. These systems work at the granularity of jobs and machines, where decisions are taken every few seconds and a costly model can be afforded.

Inside the kernel the time budget is much smaller. Chen et al. [14] trained a small multilayer perceptron to imitate the task migration decisions of the Linux load balancer and ran it in the kernel with low overhead, which demonstrates that learned models can be placed on the scheduling path if they are kept small. Zhang and Huang [4] discussed the wider vision of a learned operating system and pointed out open issues such as model overhead, the lack of training data inside the kernel and the need for safe fallbacks.

E. Workload Traces and Models

Reiss et al. [15] analysed a publicly released cluster trace and showed that task durations and resource demands are highly heterogeneous, with many short tasks and a few very long ones. Feitelson et al. [16] described the Parallel Workloads Archive and the precautions needed when using recorded logs for scheduler evaluation. For the predictor itself, gradient boosted trees [17] offer fast inference on tabular features, while long short-term memory networks [18] are designed for sequences and can model how the bursts of a process evolve over time. Table I summarizes the surveyed work.

TABLE I. SUMMARY OF THE LITERATURE SURVEY

Ref.	Area	Main idea	Limitation for this problem
[1]	Real-time scheduling	Utilization bounds for fixed and dynamic priority	Assumes known periodic tasks
[2]	Proportional share	Randomized allocation using tickets	Does not adapt slice length to bursts
[3]	Linux scheduler	Reports bugs caused by complex heuristics	Diagnosis only, no adaptive policy
[5]	Dynamic RR	Quantum from median burst of ready queue	Burst times assumed known
[6]	Dynamic RR	Median quantum with queue rearrangement	Burst times assumed known; tiny test sets
[7]	Dynamic RR	Quantum from mean of remaining bursts	Burst times assumed known
[10]	ML in Linux	Predicts CPU need of a program from its attributes	Per program, not per burst
[8]	HPC backfilling	Run time predicted from recent jobs of the user	Batch jobs, coarse time scale
[9]	HPC backfilling	Online regression with asymmetric loss	Batch jobs, coarse time scale
[11], [12]	Cluster scheduling	Reinforcement learning policies	Too heavy for per-dispatch decisions
[13]	Cluster management	Classification of workload requirements	Targets resource allocation, not quantum
[14]	In-kernel ML	Small neural network for load balancing	Does not address time quantum
[4]	Learned OS	Vision and open challenges	No implementation or evaluation
[15], [16]	Workload traces	Characterization of real workloads	Data sources, not scheduling methods

VII. RESEARCH GAP

The survey leads to the following gaps.

- Dynamic quantum RR algorithms [5], [6], [7] depend on burst times that are not available to a real scheduler. Their reported gains have not been checked with predicted burst times.
- Prediction work in scheduling [8], [9] concentrates on the total run time of batch jobs. Prediction of individual CPU bursts of interactive and mixed processes has received much less attention.
- Learning based schedulers [11], [12], [13] operate at cluster level, and the in-kernel study [14] targets load balancing. A learned model that drives the choice of time quantum has not been examined.
- Evaluations of dynamic RR use very small process sets, and the cost of making a prediction at every dispatch is usually ignored, although it is one of the main concerns raised for learned operating systems [4].

The project aims to fill these gaps by joining the two lines of work: a burst predictor that uses only run-time information, and a dynamic quantum policy that consumes its output.

VIII. PROPOSED SOLUTION AND METHODOLOGY

A. Overview

The proposed scheduler is called burst-aware adaptive round robin (BARR). For every process that enters the ready queue, a predictor estimates the length of its next CPU burst. A quantum controller turns this estimate into a quantum that is long enough for the burst to finish in one turn when the prediction is correct, but bounded so that a wrong prediction cannot let one process hold the processor for too long. When the burst ends, its measured length is fed back to update the history of that process.

B. Proposed Architecture

Fig. 1 shows the block diagram. Traces from three sources are converted to a common format of arrival time, burst length and process identity. The feature extraction block builds a feature vector for each burst. The predictor, the quantum controller and the dispatcher form the scheduler, and the metrics collector records the events produced by the simulated CPU.

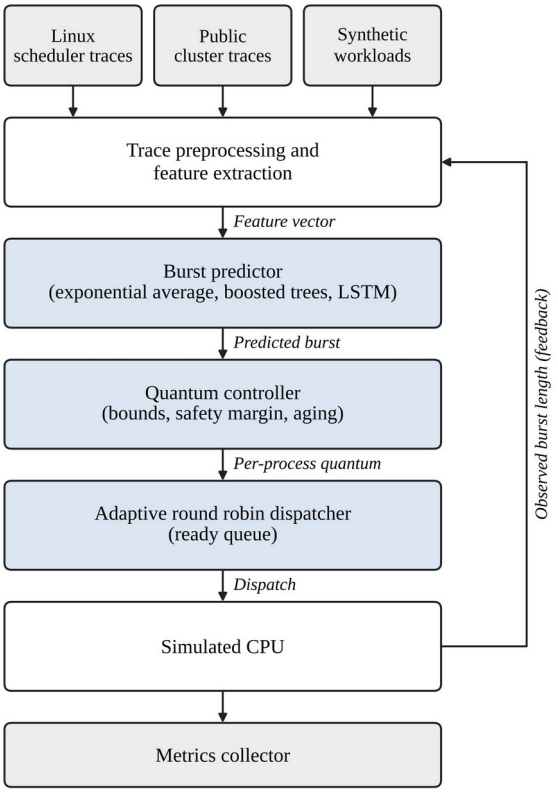


Fig. 1. Block diagram of the proposed burst-aware adaptive round robin scheduler.

C. Features and Predictor Models

The planned features are the lengths of the last k bursts of the process, their mean and variance, the current exponential average from (1), the time spent blocked before becoming ready, the fraction of earlier quanta that ended in preemption, and the static priority. Three predictors will be compared: exponential averaging as the baseline, gradient boosted regression trees [17] on the tabular features, and a small long short-term memory network [18] on the raw burst sequence. Prediction error will be measured by the mean absolute error

$$\mathrm{MAE} = (1 / N) \Sigma \mid t_i - p_i \mid \qquad (2)$$

where t_i is the actual length of burst i, p_i is its predicted length and N is the number of bursts in the test set.

D. Quantum Selection

The quantum given to process i is computed as

$$q_i = \min(q_{\max}, \max(q_{\min}, (1 + \delta)\, p_i)) \qquad (3)$$

where δ is a safety margin that absorbs small under-predictions, and q_{min} and q_{max} are lower and upper bounds. The upper bound preserves the response time guarantee of RR, and the lower bound prevents excessive switching.

E. Initial Algorithm

The initial workflow of the scheduler is given below. It will be refined during implementation.

Step 1: When a process becomes ready, build its feature vector from the stored burst history.

Step 2: Obtain the predicted burst p_i from the predictor. If the process has fewer than k recorded bursts, use the exponential average in (1), starting from a default value.

Step 3: Compute the quantum q_i from (3) and insert the process at the tail of the ready queue.

Step 4: Dispatch the process at the head of the queue and run it for at most q_i.

Step 5: If the burst completes within the quantum, record its actual length and update the history.

Step 6: If the quantum expires first, preempt the process, increase its predicted remaining burst, increment its aging counter and return it to the tail of the queue.

Step 7: If the aging counter of any waiting process exceeds a threshold, move that process to the head of the queue so that it cannot starve.

Step 8: Repeat from step 4 until the trace is exhausted, then report the metrics.

F. Dataset and Source of Data

No ready-made data set of labelled CPU bursts is known to us, so the data will be assembled from three sources. First, scheduler events will be recorded on a Linux machine with the kernel tracing facilities while running a mix of interactive, compilation and compute-bound programs; the interval between a switch-in and the following switch-out of a thread gives one burst. Second, task records from a public cluster trace [15] and logs from the Parallel Workloads Archive [16] will be used to test the scheduler on heavy-tailed duration distributions. Third, synthetic workloads with controlled burst distributions will be generated to study specific cases such as purely short or purely long processes. The bursts of each process will be split chronologically, with the earlier part used for training and the later part for testing, so that the predictor is never trained on future behavior.

G. Evaluation Parameters

- Average waiting time and average turnaround time.
- Average response time and its 95th percentile.
- Number of context switches and throughput.
- Prediction accuracy by (2) and the share of bursts that are under-predicted.
- Inference time per prediction, to judge whether the model is practical on the dispatch path.
- Maximum waiting time of any process, as a check on fairness and starvation.

The baselines are first come first served, shortest job first with exponential averaging, RR with several fixed quanta, and the dynamic quantum algorithms of [5], [6] and [7]. The dynamic algorithms will be run twice, once with the true burst times as in the original papers and once with predicted burst times.

IX. Expected Outcomes

- A reusable data set of per-process CPU bursts with the tools used to collect it.
- A comparison showing how far learned predictors can reduce burst prediction error relative to exponential averaging.
- A measurement of how much of the improvement claimed for dynamic quantum RR is retained when burst times are predicted instead of assumed.
- A scheduler that is expected to need fewer context switches than fixed quantum RR at similar response time, with bounded waiting time for every process.
- An estimate of the prediction overhead, indicating whether the approach could later be moved into a kernel.

These are expectations only. They will be confirmed or revised with experimental results in the second review.

References

[1] C. L. Liu and J. W. Layland, “Scheduling algorithms for multiprogramming in a hard-real-time environment,” J. ACM, vol. 20, no. 1, pp. 46–61, Jan. 1973.

[2] C. A. Waldspurger and W. E. Weihl, “Lottery scheduling: Flexible proportional-share resource management,” in Proc. 1st USENIX Symp. Operating Systems Design and Implementation (OSDI), 1994, pp. 1–11.

[3] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, “The Linux scheduler: A decade of wasted cores,” in Proc. 11th Eur. Conf. Computer Systems (EuroSys), 2016, Art. no. 1.

[4] Y. Zhang and Y. Huang, “‘Learned’: Operating systems,” ACM SIGOPS Oper. Syst. Rev., vol. 53, no. 1, pp. 40–45, Jul. 2019.

[5] R. J. Matarneh, “Self-adjustment time quantum in round robin algorithm depending on burst time of the now running processes,” Amer. J. Appl. Sci., vol. 6, no. 10, pp. 1831–1837, 2009.

[6] H. S. Behera, R. Mohanty, and D. Nayak, “A new proposed dynamic quantum with re-adjusted round robin scheduling algorithm and its performance analysis,” Int. J. Comput. Appl., vol. 5, no. 5, pp. 10–15, Aug. 2010.

[7] A. Noon, A. Kalakech, and S. Kadry, “A new round robin based scheduling algorithm for operating systems: Dynamic quantum using the mean average,” Int. J. Comput. Sci. Issues, vol. 8, no. 3, pp. 224–229, May 2011.

[8] D. Tsafrir, Y. Etsion, and D. G. Feitelson, “Backfilling using system-generated predictions rather than user runtime estimates,” IEEE Trans. Parallel Distrib. Syst., vol. 18, no. 6, pp. 789–803, Jun. 2007.

[9] É. Gaussier, D. Glesser, V. Reis, and D. Trystram, “Improving backfilling by using machine learning to predict running times,” in Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC), 2015, Art. no. 64.

[10] A. Negi and P. K. Kumar, “Applying machine learning techniques to improve Linux process scheduling,” in Proc. IEEE Region 10 Conf. (TENCON), 2005, pp. 1–6.

[11] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in Proc. 15th ACM Workshop Hot Topics in Networks (HotNets), 2016, pp. 50–56.

[12] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in Proc. ACM SIGCOMM Conf., 2019, pp. 270–288.

[13] C. Delimitrou and C. Kozyrakis, “Quasar: Resource-efficient and QoS-aware cluster management,” in Proc. 19th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2014, pp. 127–144.

[14] J. Chen, S. S. Banerjee, Z. T. Kalbarczyk, and R. K. Iyer, “Machine learning for load balancing in the Linux kernel,” in Proc. 11th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys), 2020, pp. 67–74.

[15] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and dynamicity of clouds at scale: Google trace analysis,” in Proc. 3rd ACM Symp. Cloud Computing (SoCC), 2012, Art. no. 7.

[16] D. G. Feitelson, D. Tsafrir, and D. Krakov, “Experience with using the Parallel Workloads Archive,” J. Parallel Distrib. Comput., vol. 74, no. 10, pp. 2967–2982, Oct. 2014.

[17] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794.

[18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Comput., vol. 9, no. 8, pp. 1735–1780, Nov. 1997.